```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.0;

contract StudentData {
    struct Student {
        string name;
        uint rollno;
    }

    Student[] public studentArr;

    // mapping to track whether a roll number already exists (saves gas vs looping every time)
    mapping(uint => bool) private rollExists;
    // mapping rollno -> index+1 in studentArr (0 means not present)
    mapping(uint => uint) private rollToIndex;

    event StudentAdded(string name, uint rollno);

    /// @notice Add a student if roll number doesn't already exist
    function addStudent(string memory name, uint rollno) public {
        require(!rollExists[rollno], "Student with roll no already exists");
        studentArr.push(Student(name, rollno));

        uint idx = studentArr.length - 1;
        rollExists[rollno] = true;
        rollToIndex[rollno] = idx + 1; // store index+1 so 0 means "not found"

        emit StudentAdded(name, rollno);
    }

    /// @notice Returns all students (beware: can be expensive if array grows large)
```

```
    function displayAllStudents() public view returns (Student[] memory) {
        return studentArr;
    }

    /// @notice Get student by array index (proper require condition)
    function getStudentByIndex(uint index) public view returns (Student memory) {
        require(index < studentArr.length, "Index out of bound");
        return studentArr[index];
    }

    /// @notice Get student by roll number (more efficient than scanning array)
    function getStudentByRoll(uint rollno) public view returns (Student memory) {
        uint stored = rollToIndex[rollno];
        require(stored != 0, "Student with this roll no does not exist");
        uint idx = stored - 1;
        return studentArr[idx];
    }

    function getLengthOfStudents() public view returns (uint) {
        return studentArr.length;
    }
}
```